

VXLAN implementation: result and stopping point

23 September 2026. The performance investigation is stopped at the user's request after the final candidate failed the agreed concurrent-throughput target. The implementation is not release-ready. No commit or push was made.

Decision

The requirement was at least 200–300 complete encrypted edges/s, preserving current functionality and isolation, with setup and teardown occurring concurrently. Neither the integrated implementation nor the last private-namespace candidate establishes that result. Do not deploy the private-namespace candidate or use the earlier endpoint benchmarks as a prediction of Nullnet throughput.

The uncommitted implementation work remains available for review. The unsuccessful private-namespace host-link pool, its benchmark injection hooks and temporary profiling prints have been removed from the product working tree. This report records outcomes and limits; it does not recommend discarded experimental configurations.

What the actual application achieved

These tests used the real proxy, TLS control channel, server and clients on Linux hosts 192.168.1.103 and .104. Each cold wave issued 1,008 successful HTTP requests with concurrency 32, producing 1,008 complete ingress edges. Containers were already running. The fixture contained 12 services and a separate 19-dependency topology check. Baseline and implementation used the same raised file-descriptor limit, 65,536. The baseline was built from commit e3e228b; the measured implementation client SHA-256 starts f3686f719155.

Application workload	Requests / errors	Full wave	Complete edges/s
Baseline, mixed placement	1,008 / 0	82.561 s	12.21
Implementation, mixed placement	1,008 / 0	14.769 s	68.25
Implementation, cross-host only	1,008 / 0	11.068 s	91.08
Implementation, same-host only	1,008 / 0	28.094 s	35.88

The mixed workload improved substantially, but placement was scheduler-selected, not an identical allocation trace: 553 edges crossed hosts before and 523 afterward. Therefore the approximately 5.6-fold improvement describes these observed waves, not a perfectly controlled topology-only ratio. The pure same-host and cross-host rows are separate implementation measurements, not matched before/after pairs.

Every wave recorded 2,016 successful endpoint setups and 2,016 successful endpoint teardowns across the two hosts, with no lifecycle errors and no active owned links after cleanup. The fixture's ten-second idle grace allows early edges to retire while longer cold waves are still creating later edges. These are application-ready wave timings; they are not isolated teardown-burst rates. Endpoint timing distributions and exact counts are in results.json.

Final candidate: private namespace with reusable host links

The last test placed bridges, transport links and MACsec in a private network namespace, while retaining a stable host-facing link for every bridge. Both endpoint halves kept their host-side bridge IP behavior; one endpoint also had its container attachment. A pool of 2,048 bridge/link slots was allocated before timing. The actual Rust setup and retirement paths ran with 32 setup slots and bounded native workers; retirement submissions used concurrency 192 and the existing bounded cleanup worker.

Seven sequential waves each created and retired 192 complete same-host encrypted edges. This exceeds the pool size in endpoint leases and exercises actual slot reuse. Three further waves each created 192 edges while retiring another 192. The old root-namespace bridge pool had been removed before the test. RPC, database, proxy and HTTP processing were excluded from these measurements.

Final sustained kernel test	Measured result
Pool initialization, reported separately	23.497 s
First sequential setup / teardown	207.26 / 211.07 edges/s
Subsequent six setup waves	155.74–191.75 edges/s

Final sustained kernel test	Measured result
Subsequent six teardown waves	154.91–189.43 edges/s
Concurrent wave 1: 192 creates + 192 retirements	2.787 s; 68.90 creates/s and 68.90 retirements/s
Concurrent wave 2: 192 creates + 192 retirements	2.838 s; 67.66 creates/s and 67.66 retirements/s
Concurrent wave 3: 192 creates + 192 retirements	2.854 s; 67.26 creates/s and 67.26 retirements/s

Creation and retirement rates must not be added and reported as newly created edges/s. The final candidate fails the target in the same-host case before application overhead is introduced. Full product integration of this candidate was therefore stopped. Shorter preallocation experiments had produced higher rates, but they did not establish sustained concurrent throughput or safe repeated reuse; they are not the acceptance result.

The reuse test passed its assertions: host interface indices stayed unchanged, idle links were down, their addresses and neighbors were cleared, endpoint bookkeeping returned empty, and retirement also survived unrelated link churn. These checks do not establish crash recovery or full product equivalence.

What the investigation establishes

The kernel and host environment account for a substantial part of the gap. Earlier matched runs of the actual Rust lifecycle code with a 2,048-bridge pool took 0.674 s to create 192 encrypted pairs in isolation, versus 4.128 s in the host namespace. This difference was reproduced outside the RPC/database path. It is incorrect to attribute the entire gap to Nullnet orchestration.

Private namespaces remove much of the device-creation work from host observers. Preserving existing host routing still requires host-visible link/address changes and reset work. Shared kernel operations, host notifications and their consumers remain involved. The final measurements show that retaining interfaces alone did not remove enough cost. They do not apportion every remaining millisecond to udev, NetworkManager or another particular daemon, and do not prove that Linux cannot ever reach the target.

The earlier 600–1,000/s figures count endpoint operations in a different execution environment. A complete same-host edge has two endpoint halves, and sustained churn must also retire old edges. Endpoint rates, complete-edge rates, and creation-plus-retirement operation counts are different quantities. They cannot be substituted for each other. High throughput also does not establish a UI edge-creation latency below 10 ms.

Full pool destruction remains expensive: removing the 2,048 old root bridges took 66.762 s immediately before the final test. Normal pooled retirement excludes that maintenance operation; the retained implementation still deletes overflow bridges when its idle pool cap is exceeded.

Correctness evidence and its boundary

The retained root-namespace implementation passed the Linux CI command set and real two-host cold workloads, including cleanup and restart-preservation checks. The final cleanup also removes temporary instrumentation; final CI is recorded in `verification.json`. A separate complex-topology check and the authenticated wrong-VNI test passed. This does not complete the project's entire release checklist.

Separate physical-host private-namespace packet tests passed bidirectional full-size overlay pings, wrong-VNI rejection (20/20), captured ESP replay rejection (20/20), wrong-key rejection with the unrelated edge surviving, and ciphertext-only failure when private crypto state was removed. Fragmented outer ESP delivered 20/20 authenticated frames. Lowering the prototype underlay link MTU produced a correct PMTU notification and allowed smaller packets.

Those packet tests and the repeated-pool benchmark were separate tests. External-router ICMP/PMTU forwarding, unencrypted VXLAN coexistence with Swarm, private-pool restart recovery, full egress/SNAT behavior and integrated sustained application throughput were not all proven together. The private candidate is therefore not a fully validated implementation even independently of its performance failure.

Remaining work and next decision

1. Review the retained, uncommitted native-Netlink/crypto, Docker generation-cache, global-policy, bridge-pool and acknowledged-cleanup implementation on its measured merits. It improves the current application workload but does not satisfy the requested target. Do not treat it as ready to merge.
2. If retaining those changes, finish the remaining release gates in a separate work session: Docker generation changes, complete restart and partial-failure paths, egress/host-service behavior, firewall recovery and packet isolation, then an integrated four-case before/after comparison. Complete setup documentation and a changelog entry only for the accepted release scope.
3. If 200–300 concurrent complete edges/s remains mandatory, the next investigation needs an explicit decision about the host-attachment and routing architecture. Preserving a host-visible per-edge interface still has a measured cost. Any broader redesign must first prove routing and isolation parity, then pass a repeated mixed-churn kernel test before expensive product integration. No replacement design is selected or authorized by this report.
4. Do not resume performance experiments automatically. The user requested this stopping point after the final candidate failed.

There is no honest, fully tested “after” implementation meeting the target for all four cases to report. The earlier four-case PDFs remain valid endpoint-harness results, not a completed integrated acceptance report. The current application measurements and final stopping evidence are recorded here rather than filling missing cases with estimates.

Commit scope and final lab state

Commit only the report updates if retaining the investigation record. Do not commit the implementation as a finished optimization: it missed the target and has outstanding release checks. The unfinished code remains uncommitted and has a separate recovery archive outside the repository. No failed private-namespace implementation is selected.

The lab is restored to the pinned e3e228b baseline binaries. Temporary fixture containers, stack configuration, source addresses, routes, bridge pools and private-namespace resources were removed. Original application containers were not restarted. The raised runtime file-descriptor limit remains in place; `verification.json` records the exact binaries and resource audit. The original dirty remote checkout was preserved.

References

- [Single entry point and next steps](#)
- [Machine-readable results and endpoint timing distributions](#)
- [Verification and final lab state](#)
- [Original four-case endpoint breakdown](#)
- [Change-by-change rationale](#)
- [Bare VXLAN tuning reference](#)